# Practical Machine Learning: Sessions 8–16 Cheatsheet

## (Bayesian, Regression, SVM, Time Series, Recommenders) – MCQ-Friendly

---

## Session 8 & 9: Bayesian Analysis, Naive Bayes, LDA, KNN

### Bayes Theorem & Core Concepts

**Bayes Theorem:**

$$P(C_k \mid X) = \frac{P(X \mid C_k)\, P(C_k)}{P(X)}$$

- $P(C_k)$: **prior** – belief before data.
- $P(X \mid C_k)$: **likelihood** – how likely data if class is $C_k$.
- $P(C_k \mid X)$: **posterior** – belief after data.
- Classification: $\hat{C} = \arg\max_k P(X \mid C_k)\, P(C_k)$ (ignore $P(X)$, constant across classes).

### Naive Bayes Classifier

**Key assumption:** Features are **conditionally independent** given class.

$$P(X \mid C_k) = \prod_{j=1}^{n} P(x_j \mid C_k)$$

**Types:**

- **Bernoulli**: Binary features (word present/absent).
- **Multinomial**: Word counts (text classification).
- **Gaussian**: Continuous features, normal distribution.

**Calculation example:**

- $\text{Score}(C) \propto P(x_1 \mid C) \times P(x_2 \mid C) \times \cdots \times P(C)$.
- Pick class with highest score.

**MCQ Tips:**

- "Naive" = feature independence assumption.
- $P(X)$ is ignored in practice (constant factor).

## Linear Discriminant Analysis (LDA)

**Assumes:** Each class is Gaussian, all classes share **same covariance matrix**.

**Goal:** Find **linear combination** of features that:

- Maximizes **between-class variance** (class means far apart).
- Minimizes **within-class variance** (tight class clusters).

**Method:**

- Compute within-class scatter $S_W$ and between-class scatter $S_B$.
- Find eigenvectors of $S_W^{-1} S_B$; use top eigenvectors as discriminant directions.
- Classification: compute discriminant function(s) per class → pick highest.

**Key facts:**

- **Not** assumption: same mean for all classes (that would break discrimination).
- Decision boundary: **linear** (hyperplane).
- Dimensionality reduction to ≤ (# classes − 1) dimensions.

## K-Nearest Neighbors (KNN)

**Lazy learner:** Store training data; compute at prediction time.

**Algorithm:**

1. Choose $k$.
2. Compute distance to all training points (Euclidean, Manhattan, etc.).
3. Pick k closest points.
4. **Majority voting** → predicted class.

**Properties:**

- **Non-parametric**: no explicit model, flexible decision boundary.
- **Lazy**: no training, but slow prediction for large $n$.
- Sensitive to: scale, choice of $k$, distance metric.
- Increasing $k$ → **lower variance, higher bias**.

**MCQ Tips:**

- Small $k$ → complex boundary, overfitting.
- Large $k$ → smooth boundary, underfitting.
- Always **normalize/scale** features (different units).

## Quick Comparison

| Method | Type | Boundary | Training | Typical use |
|---|---|---|---|---|
| Naive Bayes | Probabilistic | Often linear | Fast (priors) | Text, high-dim, spam |
| LDA | Linear classifier | Linear | Moderate | Balanced classes, 2–3 D |
| KNN | Non-parametric | Non-linear | Lazy (store) | Small/medium datasets |

---

## Session 10 & 11: Linear, Logistic, Polynomial, Ridge, Lasso, Elastic Net

### Linear Regression

**Loss:** Minimize **Sum of Squared Errors (SSE)** (OLS).

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

**Assumptions:** Linear relation, Gaussian errors, independent residuals, constant variance.

**MCQ Tips:**

- Uses **SSE** loss, not MAE.
- No penalty on coefficients.

### Logistic Regression

**For:** Binary (0/1) **classification**, not continuous targets.

**Maps** linear combination to probability via **sigmoid**:

$$p = \frac{1}{1 + \exp(-(z))}, \quad z = \beta_0 + \beta_1 x_1 + \dots$$

**Loss: Log-loss (cross-entropy)**, not SSE.

**Decision boundary:** Linear in log-odds space; **sigmoid** makes it S-shaped in probability.

### Polynomial Regression

**Linear in parameters** $\beta$, but uses **polynomial features** of inputs:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

**Still a linear model**, just nonlinear in $x$; high degree → overfitting risk.

## Regularization: Ridge, Lasso, Elastic Net

**General form:**

$$\text{Loss} = \text{SSE} + \lambda \cdot \text{Penalty}(\beta)$$

- $\lambda$ = regularization strength; larger $\lambda$ → more shrinkage.

**Ridge (L2):**

- Penalty: $\lambda \sum_j \beta_j^2$.
- Shrinks all coefficients **smoothly toward zero**, none exactly zero.
- Helps with **multicollinearity**, keeps all features.
- Interpretation: coefficients become more stable with correlated predictors.

**Lasso (L1):**

- Penalty: $\lambda \sum_j |\beta_j|$.
- Some coefficients become **exactly zero → automatic feature selection**.
- Useful when many features; creates sparse, interpretable models.

**Elastic Net (L1 + L2):**

- Penalty: $\lambda_1 \sum_j |\beta_j| + \lambda_2 \sum_j \beta_j^2$ (or weighted combination).
- **Sparsity** from L1 + **stability** from L2.
- Best for many correlated features; balances feature selection and stability.

**Effect of increasing $\lambda$:**

- Bias ↑, Variance ↓ (more regularization → simpler model, higher bias).

## Comparison

| Method | Penalty | Effect | Best for |
|---|---|---|---|
| OLS/Linear | None | Risk overfitting | Baseline, few features |
| Ridge | L2 | Smooth shrinkage | Multicollinearity, keep all feats |
| Lasso | L1 | Exact zeros (sparse) | Feature selection, many feats |
| Elastic Net | L1+L2 | Sparse + stable | Correlated features, balanced |

---

# Session 12: Support Vector Machines (SVM)

## Core Idea: Max-Margin Classification

**Goal:** Find hyperplane $w^\top x + b = 0$ that:

- Separates classes.
- **Maximizes margin** = distance to closest points (support vectors).

**Support vectors:** Points on margin boundaries or closest to hyperplane; alone determine decision boundary.

**Margin width:** $2/\|w\|$; maximize margin ≈ minimize $\|w\|^2$.

## Hard vs Soft Margin

- **Hard-margin SVM**: Assumes perfect separability, no misclassification.
- **Soft-margin SVM**: Allows margin violations using slack variables; penalty parameter $C$ balances margin vs misclassification.
  - Large $C$ → fewer violations, smaller margin (overfitting risk).
  - Small $C$ → more violations, larger margin (underfitting risk).

## Linear SVM

**Decision function:**

$$\hat{y} = \text{sign}(w^\top x + b)$$

**Key facts:**

- **Max-margin** classifier based on **Structural Risk Minimization (SRM)**.
- Only support vectors affect $w$ and $b$; distant points irrelevant.
- Effective in high-dimensional spaces (e.g., text).

## Kernel Trick & Nonlinear SVM

**Problem:** Data not linearly separable in original space.

**Solution:** Implicitly map to higher-dimensional space via **kernel function**:

$$K(x_i, x_j) = \text{inner product in high-dim space}$$

**Common kernels:**

**1. Polynomial Kernel:**

$$K(x, z) = (\gamma x^\top z + r)^d$$

- $d$ = degree (2, 3, etc.).
- Creates **polynomial decision boundaries** in original space.
- Parameters: $C, d, \gamma, r$.

**2. RBF (Gaussian) Kernel:**

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

- Highly **flexible, smooth, nonlinear** boundaries.
- $\gamma$ controls reach of influence (high $\gamma$ → local, low → global).
- Parameters: $C, \gamma$.

### Comparison of SVM Kernels

| Kernel | Boundary | Usage | Parameters |
|---|---|---|---|
| Linear | Hyperplane | High-dim, roughly linear (text, NLP) | $C$ |
| Polynomial | Polynomial curve ($d$) | Moderate nonlinearity | $C, d, \gamma, r$ |
| RBF | Highly flexible curve | Complex boundaries, local influence | $C, \gamma$ |

**MCQ Tips:**

- SVM = **max-margin** classifier.
- Kernel trick = implicit high-dim mapping without explicit computation.
- RBF → flexible, localized boundaries.
- Polynomial → polynomial boundaries in original space.

---

## Session 13 & 14: Time Series, Smoothing & ARIMA

### Moving Average & Exponential Smoothing

**Simple Moving Average (k-period):**

$$\hat{Y}_{t+1} = \frac{Y_t + Y_{t-1} + \cdots + Y_{t-k+1}}{k}$$

- Equal weights on last $k$ observations; reduces noise.

**Simple Exponential Smoothing (SES):**

$$\hat{Y}_t = \alpha Y_t + (1 - \alpha)\hat{Y}_{t-1}, \quad 0 < \alpha < 1$$

- Recent values have **more weight** (exponential decay).
- For series with **no trend, no seasonality**.
- $\alpha$ = smoothing parameter (high → recent data matters more).

## Holt's Trend Method (Double Exponential Smoothing)

**Components:**

- Level $L_t$.
- Trend $T_t$.

**Forecast:** $\hat{Y}_{t+h} = L_t + hT_t$.

**Use:** Series with **trend, no seasonality**.

## Holt–Winters (Triple Exponential Smoothing)

**Components:**

- Level $L_t$.
- Trend $T_t$.
- Seasonal $S_t$.

**Two flavors:**

- **Additive:** $Y_t = \text{trend} + \text{seasonal} + \text{noise}$; constant seasonal amplitude.
- **Multiplicative:** $Y_t = \text{trend} \times \text{seasonal} \times \text{noise}$; seasonal effect scales with level.

**Use:** Series with **seasonality (with or without trend)**.

**Seasonal differencing:** $Y'_t = Y_t - Y_{t-m}$ (remove periodic pattern; $m$ = season length).

## Autocorrelation, ACF, PACF

**Autocorrelation:** Correlation between $Y_t$ and past $Y_{t-k}$ (lag $k$).

**ACF (Autocorrelation Function):**

- Autocorrelation at many lags.
- Detect **MA order** and seasonality (spikes at seasonal lags).

**PACF (Partial ACF):**

- Correlation between $Y_t$ and $Y_{t-k}$ after removing intermediate lags.
- Detect **AR order** (how many past terms directly matter).

## AR, MA, ARMA, ARIMA

**Auto-Regressive AR(p):**

$$Y_t = \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + \varepsilon_t$$

- Depends on **own past values**; PACF cuts off at lag $p$.

**Moving Average MA(q):**

$$Y_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

- Depends on **past errors**; ACF cuts off at lag $q$.

**ARMA(p,q):**

$$Y_t = \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \ldots$$

- Combines AR and MA; for **stationary** series.

**ARIMA(p,d,q):**

- ARMA after $d$ **differencing** steps to achieve stationarity.
- $d = 0$ → ARMA; $d > 0$ → difference(s) then ARMA.
- $p$ = AR order, $d$ = differencing order, $q$ = MA order.

**Selection workflow:**

1. Check stationarity (plot, ACF, differencing if needed).
2. Use ACF/PACF to estimate $p$, $q$.
3. Fit ARIMA, validate diagnostics.

### Comparison

| Method | Components | Use |
|---|---|---|
| SMA | Constant avg | Noise reduction, no trend/season |
| SES | Level only | No trend, no seasonality |
| Holt's | Level + Trend | Trend, no seasonality |
| Holt–Winters | Level + Trend + Seasonal | Seasonality (± trend) |
| AR(p) | Past values | Stationary, autocorrelated series |
| MA(q) | Past errors | Stationary, uncorrelated residuals |
| ARMA(p,q) | AR + MA | Stationary mixed process |
| ARIMA(p,d,q) | AR + diff + MA | Non-stationary (with differencing) |

---

# Session 15 & 16: Recommendation Systems & Evaluation Metrics

## Recommender Systems Overview

**Goal:** Predict which items a user will **like**.

**Data:** User–item interactions (ratings, clicks, purchases, watch time).

**Approaches:**

- **Content-based**: Items similar to user's past likes.
- **Collaborative filtering**: Users' past similarity predicts future; item similarity.

## Collaborative Filtering (CF)

**Core idea:** Users who behaved similarly in past → similar future behavior.

**User-based CF:**

- Find **similar users** (cosine, Pearson similarity on ratings).
- Recommend items liked by similar users but target user hasn't consumed.
- Formula: Similarity $S(u_1, u_2)$, then aggregate neighbors' ratings.

**Item-based CF:**

- Compute **item–item similarity** (co-liked by users).
- Recommend items similar to those target user already interacted with.
- Advantages: More scalable, item similarity more stable than user similarity.

## Matrix Factorization (MF)

**Idea:** Decompose sparse user–item matrix into low-rank factors.

$$R \approx PQ^\top$$

- $P_u$: $k$-dimensional **user latent vector**.
- $Q_i$: $k$-dimensional **item latent vector**.
- Predicted rating: $\hat{R}_{u,i} = P_u^\top Q_i$.

**Interpretation:**

- Latent dimensions = hidden concepts (e.g., "action" vs "romance" for movies).
- Similar users/items have similar latent vectors.

**Common:** SVD-based MF, biased MF (user/item biases), SVD++.

## Evaluation Metrics for Recommenders

**Precision@K & Recall@K (Top-K ranking)**

**Precision@K:**

$$\text{Prec@K} = \frac{\#\text{ relevant in top K}}{K}$$

→ Quality of recommendations; what % of top-K are actually good?

**Recall@K:**

$$\text{Recall@K} = \frac{\#\text{ relevant in top K}}{\#\text{ all relevant items}}$$

→ Coverage; of all good items, how many showed up in top K?

**RMSE (Rating prediction error)**

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{(u,i)}(\hat{R}_{u,i} - R_{u,i})^2}$$

→ Measures accuracy of predicted ratings (commonly used for MF).

**Mean Reciprocal Rank (MRR)**

$$\text{MRR} = \frac{1}{|U|}\sum_{u}\frac{1}{\text{rank of first relevant item}}$$

→ Focuses on **position of first good recommendation**; useful for top-1 accuracy.

**Mean Average Precision at K (MAP@K)**

- Compute **Average Precision@K** for each user (precision at each position where relevant item appears).
- Average over all users → **MAP@K**.
- Rewards ranking **all relevant items high**, not just having some.

**NDCG@K (Normalized Discounted Cumulative Gain)**

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

- **DCG@K:** Sum of gains (relevances), discounted by log of rank (penalize deep items).
- **IDCG@K:** DCG of ideal (perfect) ranking.
- Range: 0–1; higher = better; **position-sensitive** (top items weighted more).

**Key:** Penalizes relevant items appearing deep in list more than Precision/Recall.

## Comparison of Metrics

| Metric | Type | Focus | Best for |
|---|---|---|---|
| Precision @K | Set / top-K | Quality of recommendations | High-precision scenarios |
| Recall@K | Set / top-K | Coverage of good items | High-coverage scenarios |
| RMSE | Regression | Rating prediction accuracy | Rating prediction |
| MRR | Ranking | Position of first relevant item | Top-1 or first-hit |
| MAP@K | Ranking | Ranking of all relevant items | Balanced ranking |
| NDCG@K | Ranking | Position-sensitive ranking quality | When rank position matters |

### MCQ Tips

- CF = **interaction patterns**, not content.
- **User-based** → find similar users; **Item-based** → similar items.
- **Precision** = quality; **Recall** = coverage.
- **RMSE** → rating error; **NDCG** → ranking quality.
- **MRR** → first hit; **MAP@K** → all relevant hits ranked.
- **Item-based CF** more scalable (smaller item space, stable similarities).

---

## Formula Quick Reference (CCEE-style)

### Bayes & Classification

- $P(C \mid X) = \frac{P(X|C)P(C)}{P(X)}$

### Linear Models

- Linear: $\hat{y} = \beta_0 + \beta_1 x + \ldots$ (SSE loss)
- Logistic: $p = \frac{1}{1+e^{-z}}, z = \beta_0 + \beta_1 x + \ldots$ (log-loss)
- Ridge penalty: $\lambda \sum \beta_j^2$
- Lasso penalty: $\lambda \sum |\beta_j|$

### SVM

- Margin: $2/\|w\|$
- Polynomial kernel: $K(x,z) = (\gamma x^\top z + r)^d$
- RBF kernel: $K(x,z) = \exp(-\gamma \|x-z\|^2)$

### Time Series

- SES: $\hat{Y}_t = \alpha Y_t + (1-\alpha)\hat{Y}_{t-1}$
- Holt: $\hat{Y}_{t+h} = L_t + hT_t$
- AR(p): $Y_t = \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + \varepsilon_t$
- MA(q): $Y_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$

### Recommenders

- MF: $\hat{R}_{u,i} = P_u^\top Q_i$
- Prec@K: $\frac{\text{rel in top K}}{K}$
- Recall@K: $\frac{\text{rel in top K}}{\text{all rel}}$
- RMSE: $\sqrt{\frac{1}{N}\sum(\hat{R}_{u,i} - R_{u,i})^2}$

---

## Key MCQ Patterns

**Always remember:**

1. Naive Bayes = **feature independence given class**.
2. KNN = **lazy learner** (no training, distance-based).
3. LDA = **linear discriminant** boundary with Gaussian classes.
4. Regularization increasing → **bias ↑, variance ↓**.
5. Ridge = **smooth shrinkage**, Lasso = **exact zeros**.
6. SVM = **max-margin**, kernel = **implicit mapping**.
7. Time series: **ACF → MA**, **PACF → AR**.
8. Recommender metrics: **Precision/Recall = set-based**, **NDCG/MAP = ranking-based**.

---

## Exam Tips (CCEE Format)

- **40 MCQs per module**, ~1 hour, NO negative marking (so guess wisely or skip).
- **Single correct answer** per question (unless explicitly multi-select).
- Focus on **practical** understanding (formulas, when to use what, trade-offs).
- Always check: assumptions, parameter effects, comparison of methods.